

OOPs Assignment - 2

Q1) class Animal {
 void eat() {
 System.out.println("This animal eat food.");
 }
}

class Mammal extends Animal {
 void walk() {
 System.out.println("This mammal walks.");
 }
}

class Dog extends Mammal {
 void bark() {
 System.out.println("The dog barks.");
 }
}

public class MultilevelInheritance {
 public static void main (String [] args) {
 Dog d = new Dog();
 d.eat();
 d.walk();
 d.bark();
 }
}

Q2) interface Printable {
 void print();
}

interface Showable {
 void show();
}


```
class Demo implements Printable, Showable {  
    public void print() {  
        System.out.println("Printing...");  
    }  
}
```

```
    public void show() {  
        System.out.println("showing...");  
    }  
}
```

```
public class Interface {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.print();  
        d.show();  
    }  
}
```

Q3) class Person {
 String name;

Person(String name) {
 this.name = name;

void introduce() {

System.out.println("Hi, I am + name + ".");

class Student extends Person {
 String college;

Student(String name, String college) {
 super(name);
 this.college = college;


```

y
void introduce () {
    super.introduce ();
    System.out.println ("I study at " + college + ".");
}

```

```

y
public class SuperkeywordExample {
    public static void main (String[] args) {
        Student s = new Student ("Kavish",
        "MUJ");
        s.introduce ();
    }
}

```

Q4) abstract class Vehicle {

```

    String registrationNumber;
    String brand;
    int year;

```

```

    Vehicle (String registrationNumber, String brand, int year) {
        this.registrationNumber = registrationNumber;
        this.brand = brand;
        this.year = year;
    }

```

```

    void displayDetails () {
        System.out.println ("Reg No." + registrationNumber
        + ", Brand:" + brand + ", Year:"
        + year);
    }
}

```


abstract double calculateMileage();

y

class Car extends Vehicle {

int numberOfDoors;

Car(String reg, String brand, int year, int doors) {

super(reg, brand, year);

this.numberOfDoors = doors;

y

double calculateMileage() {

return 15.0 - (0.5 * numberOfDoors);

y

y

class Bike extends Vehicle {

Boolean hasYear;

Bike(String reg, String brand, int year, boolean hasYear) {

super(reg, brand, year);

this.hasYear = hasYear;

y

double calculateMileage() {

return hasYear ? 50.0 : 35.0;

y

y

class Truck extends Vehicle {

double loadCapacity;

Truck(String reg, String brand, int year, double load) {

super(reg, brand, year);

this.loadCapacity = load;

y


```

double CalculateMileage() {
    return 10.0 - (0.5 * loadCapacity);
}

public class VehicleTest {
    public static void main (String[] args) {
        Vehicle[] vehicle = {
            new Car ("UP01", "Hyundai", 2021, 4),
            new Bike ("TN09", "Honda", 2022, true),
            new Truck ("MH12", "Tata", 2020, 4)
        };
        for (Vehicle v : vehicle) {
            v.displayDetails();
            System.out.println ("Mileage" + v.calculateMileage());
            System.out.println ();
        }
    }
}

```

Q5

```

abstract class LibraryStaff {
    String name, staffID, designation;

    LibraryStaff (String name, String staffID, String designation) {
        this.name = name;
        this.staffID = staffID;
        this.designation = designation;
    }

    public String getName() { return name; }
    public String getStaffID() { return staffID; }
    public String getDesignation() { return designation; }

    public void setName (String name) { this.name = name; }
    public void setStaffID (String staffID) { this.staffID = staffID; }
}

```



```
public void setDesignation (String designation) { this.  
    designation = designation; }
```

```
abstract double calculateMonthlySalary();
```

```
class Librarian extends LibraryStaff {  
    double monthlyFixedSalary;
```

```
    Librarian (String name, String staffID, String designation,  
        double salary) {
```

```
        super (name, staffID, designation);
```

```
        this.monthlyFixedSalary = salary;
```

```
    }
```

```
    public double calculateMonthlySalary () {  
        return monthlyFixedSalary;
```

```
    }
```

```
    public double getMonthlyFixedSalary () { return monthlyFixedSalary; }
```

```
    public void setMonthlyFixedSalary (double salary) { this.  
        monthlyFixedSalary = salary; }
```

```
    }
```

```
class Assistant extends LibraryStaff {
```

```
    double hourlyRate;
```

```
    int hoursWorked;
```

```
    Assistant (String name, String staffID, String designation,  
        double rate, int hours) {
```

```
        super (name, staffID, designation);
```

```
        this.hourlyRate = rate;
```

```
        this.hoursWorked = hours;
```

```
    }
```



```
public double calculateMonthlySalary() {
    return hourlyRate * hoursWorked;
}
```

y

```
public double getHourlyRate() { return hourlyRate; }
public void setHourlyRate(double rate) { this.hourlyRate = rate; }
```

```
public int getHoursWorked() { return hoursWorked; }
public void setHoursWorked(int hours) { this.hoursWorked = hours; }
```

y

```
public class LibrarySystem {
    public static void main(String[] args) {
        Librarian lib = new Librarian("Ram", "LIB101",
            "librarian", 50000);
        Assistant asst = new Assistant("Vikash", "AST202",
            "Assistant", 200, 160);

        System.out.println(lib.getName() + " : " + lib.calculateMonthlySalary());
        System.out.println(asst.getName() + " : " + asst.calculateMonthlySalary());
    }
}
```

y

y

Q6) An Exception is an event that disrupts normal program flow, where compiler does not know how to handle the situation.

Key terms:

try : Wraps risky code.

Catch: Handles the exception.

finally: Always executes.

throw: Used to throw an exception.

throws: Declares exceptions.

```
Q7) public class NestedTry {  
    public static void main (String [] args) {  
        try {  
            try {  
                int a = 10/0;  
            } catch (ArithmeticException e) {  
                System.out.println("Inner catch " + e);  
            }  
            int [] arr = new int [2];  
            arr[5] = 100;  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Outer catch " + e);  
        }  
    }  
}
```

```
Q8) interface Payment {  
    void pay (double amount);  
    String getPaymentType();  
}  
  
class CreditCardPayment implements Payment {  
    public void pay (double amount) {  
        System.out.println("Paid " + amount + " using Credit  
        Card.");  
    }  
}
```



```
public String getPaymentType() {  
    return "Credit Card";  
}
```

```
y  
y  
class DebitCardPayment implements Payment {  
    public void pay (double amount) {  
        System.out.println ("Paid " + amount + " using  
        Debit Card.");  
    }  
}
```

```
y  
public String getPaymentType() {  
    return "Debit Card";  
}
```

```
y  
y  
public class PayPalPayment implements Payment {  
    public void pay (double amount) {  
        System.out.println ("Paid " + amount +  
        " Paypal.");  
    }  
}
```

```
y  
public String getPaymentType() {  
    return "Paypal";  
}
```

```
y  
y  
class PaymentProcessor {  
    public void processPayment (Payment payment,  
        double amount) {  
        System.out.println ("Processing payment via:"  
        + payment.getPaymentType());  
        payment.pay (amount);  
    }  
}
```

```
y  
public class Main {
```



```
public static void main (String[] args) {
    PaymentProcessor pp = new PaymentProcessor();
```

```
    pp.ProcessPayment (new CreditCardPayment(), 1000);
```

```
    pp.ProcessPayment (new DebitCardPayment(), 2000);
```

```
    pp.ProcessPayment (new PayPalPayment(), 3000);
}
```

Q9) class Quadrilateral {

```
    int x1, y1, x2, y2, x3, y3, x4, y4;
```

```
    Quadrilateral (int x1, int y1, int x2, int y2, int x3,
        int y3, int x4, int y4) {
```

```
        this.x1 = x1; this.y1 = y1;
```

```
        this.x2 = x2; this.y2 = y2;
```

```
        this.x3 = x3; this.y3 = y3;
```

```
        this.x4 = x4; this.y4 = y4;
    }
```

```
class Trapezoid extends Quadrilateral {
    double height, base1, base2;
```

```
    Trapezoid (int ... coords) {
```

```
        super (coords[0], coords[1], coords[2], coords[3],
            coords[4], coords[5], coords[6], coords[7]);
    }
```

```
    double getArea() {
```

```
        return 0.5 * (base1 + base2) * height;
    }
```



```
class Parallelogram extends Trapezoid {  
    Parallelogram(int... coords) {  
        super(coords);  
        this.base1 = 8; this.height = 4;  
    }  
    double getArea() {  
        return base1 * height;  
    }  
}
```

```
class Rectangle extends Parallelogram {  
    Rectangle(int... coords) {  
        super(coords);  
        this.base1 = 7; this.height = 3;  
    }  
}
```

```
class Square extends Rectangle {  
    super(coords);  
    this.base1 = this.height = 4;  
}
```

```
public class Shapes {  
    public static void main(String[] args) {  
        Square sq = new Square(0, 0, 0, 4, 4, 4, 4, 0);  
        System.out.println("Square Area: " +  
            sq.getArea());  
    }  
}
```


Q10)

```
class ExceptionA extends Exception { }  
class ExceptionB extends ExceptionA { }  
class ExceptionC extends ExceptionB { }
```

```
public class ExceptionTest {  
    public static void main (String[] args) {  
        try {  
            throw new ExceptionA();  
        } catch (ExceptionA e) {  
            System.out.println ("Caught ExceptionA");  
        }  
    }  
}
```